Final Exam (Part 2)

Program Design and Data Structures (1DL201)

Teachers: Johannes Borgström and Eva Darulova

2023-06-12 / 14:00-18:00

Instructions

Read and follow these instructions carefully to increase your chance of getting good marks.

- This is a closed-book exam. You may use a standard English dictionary. Otherwise, **no notes, calculators, mobile phones, or other electronic devices are allowed**.
- Write your **exam code** at the top right corner of **each page**.
- There are **14 questions**. Each question can award up to 4 points. You can obtain a maximum of **37 points**.
- Some questions ask for a justification of your answer; your justification needs to relate the information given in the question, the course content, and your answer.
- Some questions ask you to write code; this code needs to conform to the coding convention. You only need to write function documentation if the question text states so.
- Write your answers directly on the question sheet. You may use English and Swedish.
- Johannes or Eva will come to the exam hall around one hour after the beginning of the exam to answer questions.

Question 1: Types (4 points)

Consider the following function:

```
function every_second<T, S>(f: ???, lst: ???, next: ???): ??? {
    return is_null(lst)
            ? lst
            : next
            ? pair(f(head(lst)),
                every_second(f, tail(lst), false))
            : every_second(f, tail(lst), true);
}
```

What is the **most general** type signature of `every_second`, i.e. the types of `f`, `lst`, `next` and the return value? Do **not** use the `any` type.

Answer:

```
function every_second<T, S>(f:                                  ,
    lst:                                  ,
    next:                                 ):                    {
```

Question 2: Stacks (2 points)

Recall the stack data structure and its interface from the lectures:

```
export type NonEmptyStack<T> = [T, Stack<T>];
export type Stack<T> = null | [T, Stack<T>];
export function empty<T>(): Stack<T>
export function is_empty<T>(stck: Stack<T>): stck is null
export function push<T>(e: T, stck: Stack<T>): NonEmptyStack<T>
export function top<T>(stck: NonEmptyStack<T>): T
export function pop<T>(stck: NonEmptyStack<T>): Stack<T>
```

and assume the following statements in a file:

```
import { empty, is_empty, pop, push, top } from '../lib/stack';

const s0 = push(7, push(3, empty<number>()));
```

For the following expressions, state the result of the expression, or explain why the expression results in a type error:

Answer:

```
top(push(9, push(17, pop(s0))));
```

```
is_empty(push(5, pop(pop(s0))));
```

Question 3: Queues (2 points)

Recall the queue data structure and the interface of its mutable implementation:

```
function empty<T>(): Queue<T>
function is_empty<T>(q: Queue<T>): boolean
function enqueue<T>(e: T, q: Queue<T>)
function head<T>(q: Queue<T>): T
function dequeue<T>(q: Queue<T>)
```

Complete the following Jest testcases for unit testing the queue implementation. Make sure that your testing code is correctly typed.

Answer:

```
import { empty, enqueue, dequeue, head, is_empty } from '../lib/queue_array';

test('queue␣operations', () => {
    const que = empty<number>();
    enqueue(5, que);
    enqueue(8, que);
    dequeue(que);
    enqueue(3, que);
    expect(head(que)).toBe(           );

});


test('empty␣queue', () => {
    const que = empty<number>();
    enqueue(5, que);
    enqueue(8, que);









    expect(is_empty(que)).toBe(true);
});
```

Question 4: Preconditions (2 points)

Consider the following function:

```
/**
 * Detects whether a 2D array contains special characters.
 * @precondition ???
 * @param arr 2D array to search
 * @returns true if any of the input array's elements is equal to '#' or '%',
 *              false otherwise
 */
function find_special(arr: Array<Array<string>>): boolean {
    for (let i = 0; i < arr.length; i = i + 1) {
        for (let j = 0; j < arr[0].length; j = j + 1) {
            if (arr[i][j] === "#" || arr[i][j] === "%") {
                return true;
            }
        }
    }
    return false;
}
```

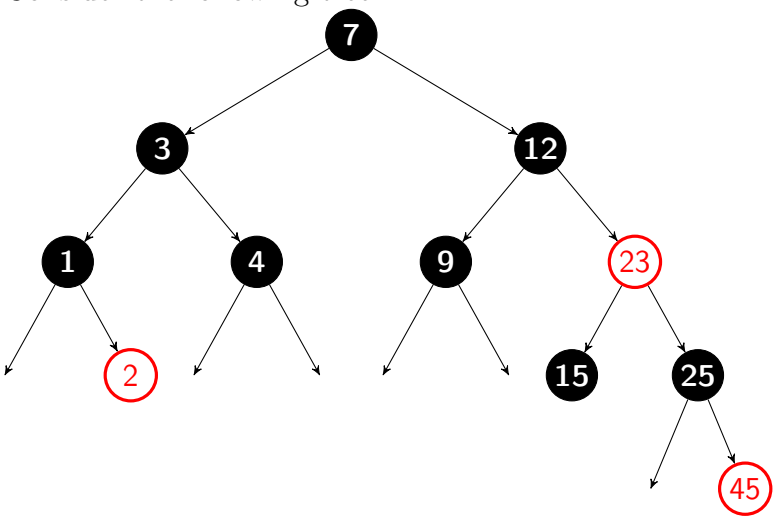
What is a suitable precondition for `find_special`? Write `true` if you think the precondition can be omitted.

Answer:

Justify your answer:

Question 5: Red-black trees (3 points)

Consider the following tree:



Is it a valid red-black tree? Justify your answer.

Answer:

Question 6: Complexity (3 points)

What is the **worst-case time** complexity in terms of Big-Theta of the following operations?

Answer:

- `dequeue` operation on a queue with n elements:

  _____

- search in a balanced binary tree with n elements:

  _____

- depth-first search in a directed acyclic graph with V nodes and E edges:

  _____

Question 7: Records (2 points)

Recall the video editing homeworks where a video was represented as a stream of frames, and each frame was represented as an array of arrays of pixels.
Assume that there is already a `Pixel` type, and define a record type for frames, containing its dimensions and the pixel data. **Answer:**

```
type Frame = {
                    :
                    :
                    :
    } ;
```

Question 8: Heaps (3 points)

Insert a new value 9 into the following array heap using the algorithm seen in class (called MAX-HEAP-INSERT in I2A), showing the contents of the array at each intermediate step.
Answer:

7	5	6	3	1	2	(input array)

							(array after last step)

Question 9: Hashtables (3 points)

Assume that you have a hash table with 4 slots that uses the hash function $h(s) = (\texttt{length}\ s)^2$ and open addressing with linear probing to resolve collisions. Let $\perp$ denote an empty cell, and Δ a deleted one.
Perform the following three operations, and show the resulting table after each of them.
`insert("lightning")`, `delete("thunder")`, `insert("very")`.

Answer:

0	1	2	3	
fandango	Δ	thunder	$\perp$	(initial table)

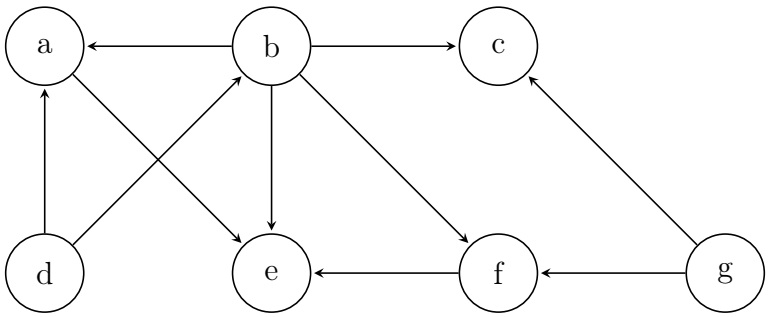
0	1	2	3	
				(after inserting "lightning")

0	1	2	3	
				(after deleting "thunder")

0	1	2	3	
				(final table)

Question 10: Graph representation (2 points)

Consider the following graph:



Represent this graph as adjacency lists.

Answer:

Question 11: Graph traversal (2 points)

Consider the graph in question 10. Perform a depth-first search in the graph, using the restart order g, f, e, d, c, b, a. What are its discovery and finish orders (i.e., in which order are the nodes coloured grey and black, respectively)?

Answer:

(discovery order)

(finish order)

Question 12: Topological sort (3 points)

Consider the graph in Question 10. Can node e be before d in a topological sort of this graph? Motivate your answer, beginning from the definition of a topological sort. **Answer:**

Question 13: Strongly connected components (3 points)

Let $G = (V, E)$ be a directed acyclic graph.
What are the strongly connected components of G? Motivate your answer.

Question 14: Lazy evaluation (3 points)

Recall that TypeScript is an applicative order language, thus the following code:

```
function unless<T>(condition: boolean, usual_value: T, exceptional_value: T): T {
    return condition ? exceptional_value : usual_value;
}
unless(true, console.log("foo"), console.log("bar"));
```

prints:

```
foo
bar
```

even though only the `exceptional_value` is really needed.
Rewrite the function `unless`, the types of its arguments, and the call to it, such that it becomes lazy and only prints `bar`.
Answer:

```
function unless<T>(condition: boolean,





}

unless(true,
```

This is the final page.